

TEXT GENERATION USING OLLAMA AND PYTHON

Question 5 – Implementation

1. Question

Demonstrate text generation using a local Large Language Model through Ollama and Python.

2. Aim

To demonstrate how a Python application can communicate with a locally running Large Language Model through Ollama and generate text from a user-provided prompt.

3. Technologies Used

- Python 3.12.10
- Ollama 0.33.2
- Llama 3.2 – local Large Language Model
- Ollama Python library
- VS Code

4. Project Structure

Local_LLM_Text_Generation/

- ├── venv/ – Python virtual environment
- ├── app.py – Question 1 text generation
- ├── summarization.py – Question 2 text summarization
- ├── qa.py – Question 3 question answering
- ├── translation_paraphrasing.py – Question 4 translation and paraphrasing
- └── ollama_text_generation.py – Question 5 Ollama and Python demonstration

5. Implementation Steps

1. Use the existing Local_LLM_Text_Generation project and virtual environment.
2. Ensure Ollama is installed and the Llama 3.2 model is available locally.
3. Import the ollama Python library.
4. Create a function to send a prompt to the Llama 3.2 model using ollama.chat().
5. Accept a prompt from the user through the Python input() function.
6. Send the prompt to the local model through Ollama.
7. Receive the generated response from the model.
8. Display the generated text in the Python terminal.
9. Run the program using python ollama_text_generation.py.

6. Python Implementation

```
import ollama

def generate_text(prompt):
    response = ollama.chat(
        model="llama3.2",
        messages=[
            {
                "role": "user",
                "content": prompt
            }
        ]
    )

    return response["message"]["content"]

print("=" * 60)
print("LOCAL LLM TEXT GENERATION USING OLLAMA")
print("=" * 60)

prompt = input("\nEnter your prompt: ")

if prompt.strip():
    print("\nGenerating response...\n")

    result = generate_text(prompt)

    print("Generated Text:")
    print("-" * 60)
    print(result)
    print("-" * 60)
else:
    print("\nPlease enter a valid prompt.")
```

7. Working Principle

The Python program accepts a prompt from the user and passes it to the Llama 3.2 model through the Ollama Python library. Ollama acts as the local model runtime and returns the generated response to the Python application. The application then prints the generated text in the terminal.

8. System Workflow

10. User enters a text-generation prompt.
11. Python receives the prompt using input().
12. The prompt is passed to the generate_text() function.
13. The function calls Ollama using ollama.chat().
14. Ollama sends the request to the locally running Llama 3.2 model.
15. Llama 3.2 generates the requested text.
16. The generated response is returned to Python.

17. Python displays the response in the terminal.

9. Input

Test prompt: Explain Machine Learning in simple words.

10. Output

The local Llama 3.2 model generated a detailed explanation of Machine Learning. The response explained that Machine Learning enables computers to learn from examples and data, identify patterns, make predictions or decisions, and improve with training. It also described supervised, unsupervised, and reinforcement learning.

11. Demonstrated Terminal Output

LOCAL LLM TEXT GENERATION USING OLLAMA

Enter your prompt: Explain Machine Learning in simple words.

Generating response...

Generated Text:

Machine Learning (ML) is a way for computers to learn and get better at doing things on their own, without being explicitly programmed.

The response further explained learning from examples, identifying patterns, making predictions or decisions, and improving with more training data. It also described Supervised Learning, Unsupervised Learning, and Reinforcement Learning.

12. Result

Thus, text generation using a local Large Language Model was successfully demonstrated through Ollama and Python. The Llama 3.2 model processed the user prompt locally and generated a meaningful response that was displayed in the terminal.

13. Conclusion

This implementation demonstrates the basic integration between Python, Ollama, and a locally running Large Language Model. It shows that prompts can be submitted directly from a Python program and processed by Llama 3.2 without requiring a cloud-based LLM service.